



پاسخ‌ها لزوماً یکتا نیستند و پاسخ‌های درست دیگر نیز در نظر گرفته می‌شوند.

۱) تعیین درستی یا نادرستی عبارات

۱) درست

دلیل: یک تابع هیوریستیک زمانی قابل قبول است که هرگز هزینه را بیش از مقدار واقعی (h^*) تخمین نزند. چون هزینه واقعی رسیدن به هدف همواره نامنفی است، مقدار صفر همواره شرط $0 \leq h^*(n)$ را ارضا می‌کند.

۲) درست

دلیل: طبق قضایای موجود در مباحث جستجوی آگاهانه، شرط یکنواختی (Consistency) قوی‌تر از قابل قبول بودن (Admissibility) است و هر تابعی که یکنواخت باشد، لزوماً قابل قبول نیز هست.

۳) درست

دلیل: از آنجا که هر دو تابع h_1 و h_2 قابل قبول هستند ($h_1, h_2 \leq h^*$)، مقدار بیشینه آن‌ها نیز قطعاً از h^* بزرگتر نخواهد بود و ویژگی قابل قبول بودن حفظ می‌شود.

۴) درست

دلیل: در روش جستجوی درختی برخلاف جستجوی گرافی، مجموعه‌ای برای نگهداری گره‌های ملاقات شده وجود ندارد، لذا یک حالت می‌تواند از مسیرهای مختلف به عنوان گره‌های مجزا تکرار شود.

۵) درست

دلیل: در این حالت برای تمامی گره‌های روی مسیر بهینه $f(n) = C^*$ و برای سایر گره‌ها $f(n) > C^*$ خواهد بود. در نتیجه الگوریتم بدون گسترش گره‌های اضافی، مستقیماً به سمت هدف حرکت می‌کند.

۶) نادرست

دلیل: در یک مسئله ساده‌شده (Relaxed) محدودیت‌ها کاهش می‌یابند، بنابراین ممکن است مسیرهای جدید و کوتاه‌تری ایجاد شوند که باعث شود مسیر بهینه مسئله اصلی، دیگر در مسئله جدید بهینه نباشد.

(۷) درست

دلیل: در جستجوی گرافی اگر هیوریستیک یکنواخت نباشد، ممکن است اولین مسیر یافته شده به یک گره بهینه نباشد. برای حفظ بهینگی در این شرایط، باید اجازه داد گره‌ها دوباره بازگشایی شوند.

(۲) اثبات و مثال نقض گزاره‌ها

در این بخش به بررسی تحلیلی گزاره‌های سوال دوم می‌پردازیم:

(۱) نادرست

مثال نقض: برای اینکه الگوریتم A^* پاسخ بهینه را تضمین کند، تابع هیوریستیک باید «قابل قبول» (Admissible) باشد، یعنی هرگز هزینه را بیش از مقدار واقعی تخمین نزند ($h(n) \leq h^*(n)$). اگر تابع ثابت ما مقدار بزرگی داشته باشد (مثلاً $h(n) = 100$)، برای گره‌هایی که فاصله واقعی آن‌ها تا هدف کمتر از ۱۰۰ است، این تابع دیگر قابل قبول نیست. در چنین حالتی، الگوریتم A^* ممکن است یک مسیر غیربهینه را انتخاب کند و بهینگی آن از بین می‌رود. بنابراین گزاره «همواره پاسخ بهینه را پیدا می‌کند» برای هر تابع ثابتی درست نیست.

(۲) نادرست

مثال نقض: فرض کنید در مسیری از A به B با هزینه یال $c(A, B) = 1$ داریم. اگر هزینه واقعی تا هدف از A برابر ۴ و از B برابر ۳ باشد، مقادیر $h(A) = 4$ و $h(B) = 1$ را در نظر بگیرید. هر دو قابل قبول هستند ($4 \leq 4$ و $1 \leq 3$)، اما شرط یکنواختی $h(A) \leq c(A, B) + h(B)$ برقرار نیست ($4 \leq 1 + 1$ نادرست است).

(۳) درست

اثبات: طبق تعریف یکنواختی $h(n) \leq c(n, n') + h(n')$ است. می‌دانیم $g(n') = g(n) + c(n, n')$ با جایگذاری در تابع f :

$$f(n') = g(n') + h(n') = g(n) + c(n, n') + h(n')$$

چون طبق فرض $c(n, n') + h(n') \geq h(n)$ است، نتیجه می‌گیریم $f(n') \geq g(n) + h(n)$ یا به عبارتی $f(n') \geq f(n)$. پس مقدار f در طول مسیر غیرکاهشی است.

(۴) درست

اثبات: شرط قابل قبول بودن یعنی $h_i(n) \leq h^*(n)$. چون $h_1(n) \leq h^*(n)$ و $h_2(n) \leq h^*(n)$ است، پس قطعاً مقدار کمینه آن‌ها نیز از هزینه واقعی بیشتر نخواهد بود:

$$\min(h_1(n), h_2(n)) \leq h_1(n) \leq h^*(n)$$

بنابراین $h(n)$ جدید نیز همواره قابل قبول است.

(۵) درست

دلیل: در فرآیند Relaxation، محدودیت‌های مسئله حذف می‌شوند. بنابراین هر مسیر یا حالتی که در مسئله اصلی وجود داشته، در مسئله ساده‌شده نیز معتبر باقی می‌ماند. علاوه بر آن، مسیرهای جدیدی که قبلاً به دلیل محدودیت‌ها ممنوع بودند نیز ممکن است اضافه شوند، لذا فضای حالت اصلی زیرمجموعه فضای حالت ساده‌شده است.

(۶) درست

دلیل: الگوریتم IDA* در هر تکرار با افزایش حد آستانه، جستجو را دوباره از گره ریشه آغاز می‌کند. بنابراین تمامی گره‌هایی که در تکرارهای قبلی بررسی شده بودند، در تکرار جدید مجدداً مشاهده و گسترش می‌یابند.

(۳) تعریف و تفاوت معیارهای ارزیابی

در این بخش به بررسی دو اصطلاح کلیدی در طراحی عامل‌های هوشمند و تفاوت‌های بنیادی آن‌ها می‌پردازیم:

• **معیار کارایی:** معیاری است که طراح برای سنجش این که عامل (Agent) چقدر در این محیط خوب عمل می‌کند، به کار می‌برد. این معیار تابعی از توالی حالت‌ها، کنش‌ها و پیامدها است.

- **مثال در بازی شطرنج:** برد ($PM = 1$)، مساوی ($PM = 0.5$) و باخت ($PM = 0$).

• **تابع ارزیابی:** تابعی است که خود عامل برای تصمیم‌گیری به کار می‌برد. این تابع به یک حالت عددی اختصاص می‌دهد که نشان‌دهنده میزان خوب بودن آن حالت از دید عامل است.

تفاوت‌های اصلی میان PM و EF

مطابق تحلیل ارائه شده، تفاوت‌های این دو مفهوم در جدول زیر خلاصه شده است:

ویژگی	معیار کارایی (PM)	تابع ارزیابی (EF)
چه کسی از آن استفاده می‌کند؟	طراح (Designer)	عامل (Agent)
در کجا اعمال می‌شود؟	تعاملات بین محیط و عامل	بر روی گره‌های جستجو
هدف	نشان‌دهنده هدف واقعی سیستم	کمک به عامل در تصمیم‌گیری
قابلیت محاسبه‌پذیری	ممکن است در انتها یا با یک خروجی مشخص شود	در طول اجرا و برای هر تصمیم باید محاسبه شود

(۴) تحلیل روش‌های جستجوی فضای حالت

در این بخش به بررسی تفاوت‌های بنیادی میان جستجوی درختی و گراف‌ی و تحلیل چالش‌های آن‌ها می‌پردازیم:

• **جستجوی درختی:** در این روش، الگوریتم بدون در نظر گرفتن این که آیا یک حالت قبلاً ملاقات شده است یا خیر، آن را به درخت جستجو اضافه می‌کند. این روش حافظه کمتری مصرف می‌کند اما نسبت به حالت‌های تکراری حساس است.

• **جستجوی گرافی:** این روش از یک مجموعه به نام Set Explored برای ذخیره تمامی حالت‌های گسترش یافته استفاده می‌کند تا از تکرار مجدد آن‌ها جلوگیری کند.

پاسخ به پرسش‌های تکمیلی

(۱) در این روش، وجود حالت‌های تکراری یا چرخه‌ها (Cycles) در فضای حالت می‌تواند باعث شود الگوریتم در یک ****حلقه بی‌نهایت**** گیر کرده یا تعداد گره‌های تولید شده به صورت نمایی افزایش یابد، حتی اگر فضای حالت فیزیکی کوچک باشد. فرض کنید در یک شبکه شطرنجی، عامل می‌تواند بین دو خانه A و B حرکت کند. در Tree-Search عامل از A به B می‌رود، سپس دوباره از B به A بازمی‌گردد و این روند به صورت $A \rightarrow B \rightarrow A \rightarrow B \dots$ در درخت جستجو تا ابد ادامه می‌یابد.

(۲) این روش پیش از گسترش هر گره، ابتدا بررسی می‌کند که آیا حالت متناظر با آن گره در Explored Set وجود دارد یا خیر. اگر حالت قبلاً مشاهده شده باشد، آن گره نادیده گرفته شده و از گسترش مجدد و ایجاد شاخه‌های تکراری جلوگیری می‌شود.

(۳) اگر هیوریستیک یکنواخت (Consistent) نباشد، ممکن است اولین باری که الگوریتم از طریق یک مسیر به حالتی می‌رسد، آن مسیر کوتاه‌ترین مسیر (بهینه) نباشد. در جستجوی گرافی، چون گره‌های ملاقات شده دوباره باز نمی‌شوند، الگوریتم ممکن است با یک مسیر غیربهینه به هدف برسد و ویژگی بهینگی (Optimality) خود را از دست بدهد. برای حل این مشکل، یا باید هیوریستیک یکنواخت باشد و یا اجازه بازگشایی مجدد (Reopening) گره‌ها داده شود.

(۵) تحلیل الگوریتم RBFS بر روی شبکه ۴ در ۴

در این بخش، روند اجرای الگوریتم Search Best-First Recursive بر اساس محاسبات دست‌نویس ارائه شده، پیاده‌سازی می‌گردد.

۱. مقادیر تابع هیوریستیک (فاصله منهن)

با فرض مختصات هدف $G(3, 0)$ و فرمول $h(n) = |x_n - x_G| + |y_n - y_G|$ ، مقادیر برای خانه‌های شبکه به شرح زیر است:

$h = 3 : (۳,۳)$	$h = 4 : (۲,۳)$	$h = 5 : (۱,۳)$	$h = 6 : (۰,۳)$
$h = 2 : (۳,۲)$	$h = 3 : (۲,۲)$	$X : (۱,۲)$	$h = 5 : (۰,۲)$
$h = 1 : (۳,۱)$	$X : (۲,۱)$	$h = 3 : (۱,۱)$	$X : (۰,۱)$
$G = 0 : (۳,۰)$	$h = 1 : (۲,۰)$	$h = 2 : (۱,۰)$	$h = 3 : (۰,۰)$

۲. روند گسترش گره‌ها و محاسبات f-value

بر اساس الگوریتم RBFS، گره‌ها با مقدار $f(n) = g(n) + h(n)$ ارزیابی می‌شوند:

- گسترش $S(0, 3)$: فرزندان: $(0, 2)$ با $f = 6$ و $(1, 3)$ با $f = 6$. گره $(0, 2)$ به عنوان بهترین انتخاب (Best) و $(1, 3)$ به عنوان جایگزین (Alternative) در نظر گرفته می‌شود.
- بررسی گره $(۰,۲)$: این گره بن‌بست است، بنابراین به S بازگشته و مقدار آن به ∞ تغییر می‌یابد.
- گسترش $(۱,۳)$: فرزند این گره $(2, 3)$ با $h = 4$ و $g = 2$ است که نتیجه می‌دهد $f = 6$.
- گسترش $(۲,۳)$: فرزندان شامل $(3, 3)$ با $f = 6$ و $(2, 2)$ با $f = 6$ هستند.

۳. ترتیب گسترش و مقادیر نهایی

مطابق با جدول استخراج شده از محاسبات:

گره	S	(۱,۳)	(۲,۳)	(۳,۳)	(۳,۲)	(۳,۱)	G
$g(n)$	۰	۱	۲	۳	۴	۵	۶
$h(n)$	۶	۵	۴	۳	۲	۱	۰
$f(n)$	۶	۶	۶	۶	۶	۶	۶

۴. مسیر نهایی

$$S \rightarrow (1, 3) \rightarrow (2, 3) \rightarrow (3, 3) \rightarrow (3, 2) \rightarrow (3, 1) \rightarrow G$$

۶) تحلیل هیوریستیک در مسیریابی خودروی امدادی

در این بخش به بررسی توابع هیوریستیک برای یافتن کوتاه‌ترین مسیر (برحسب زمان) برای یک خودروی امدادی می‌پردازیم:

(۱) **مقایسه دقت:** تابع هیوریستیک $h_2(n)$ نسبت به $h_1(n)$ دقیق‌تر است. دلیل: هر دو تابع از فاصله مستقیم استفاده می‌کنند، اما چون سرعت نور بسیار فراتر از حداکثر سرعت مجاز شهری (۱۲۰ کیلومتر بر ساعت) است، مخرج کسر در h_1 بسیار بزرگتر بوده و در نتیجه مقدار تخمینی آن بسیار کوچک و نزدیک به صفر می‌شود. از آنجا که هر دو تابع قابل قبول هستند و همواره $h_2(n) \geq h_1(n)$ برقرار است، تابع h_2 به هزینه واقعی (h^*) نزدیک‌تر بوده و آگاه‌تر محسوب می‌شود.

استفاده از یک تابع هیوریستیک ضعیف (مانند h_1 که مقادیر آن نزدیک به صفر است) باعث می‌شود الگوریتم A^* اطلاعات جهت‌دهی کمی داشته باشد و رفتاری مشابه با جستجوی هزینه یکنواخت (UCS) پیدا کند. در این حالت، الگوریتم به جای تمرکز بر مسیر مستقیم به سمت هدف، گره‌های بسیار زیادی را در تمامی جهات (به صورت دایره‌ای حول مبدأ) گسترش می‌دهد که منجر به افزایش حافظه مصرفی و زمان محاسبات می‌گردد.

(۲) **بررسی قابل قبول بودن** $h_3(n) = 2 \times h_2(n)$: خیر، این تابع لزوماً قابل قبول (Admissible) نیست.

مثال نقض: فرض کنید خودرو در بخشی از مسیر (مانند یک بزرگراه خلوت) بتواند دقیقاً با حداکثر سرعت مجاز یعنی ۱۲۰ کیلومتر بر ساعت حرکت کند. در این حالت $h_2(n)$ دقیقاً برابر با هزینه واقعی (h^*) خواهد بود. با ضرب این مقدار در ۲، مقدار هیوریستیک (h_3) از هزینه واقعی بیشتر می‌شود ($h_3 > h^*$) که این امر باعث نقض شرط قابل قبول بودن می‌گردد.

(۳) **بهترین روش ترکیب ده تابع هیوریستیک:** برای داشتن تابعی که هم «قابل قبول» باشد و هم «بیشترین دقت» را داشته باشد، باید از عملگر **Max** استفاده کرد.

فرمول:

$$h_{final}(n) = \max(h_1(n), h_2(n), \dots, h_{10}(n))$$

توضیح: از آنجا که تک‌تک این توابع قابل قبول هستند، بیشینه آن‌ها نیز هرگز از h^* تجاوز نمی‌کند و شرط بهینگی حفظ می‌شود. این ترکیب باعث می‌شود در هر نقطه، دقیق‌ترین تخمین موجود انتخاب شده و تعداد گره‌های گسترش یافته به حداقل برسد.

(۷) مسیریابی پهباد خودران و مدیریت محدودیت‌ها

در این بخش به تحلیل استراتژی‌های مسیریابی برای یک پهباد در فضای سه‌بعدی و بررسی ویژگی‌های توابع هیوریستیک مختلف می‌پردازیم:

(۱) **معرفی هیوریستیک دقیق برای حرکت ۸ جهته:** در محیطی که حرکت در ۸ جهت (افقی، عمودی و قطری) مجاز است، تابع هیوریستیک Octile Distance دقیق‌ترین تخمین را ارائه می‌دهد.

فرمول: اگر $dx = |x_n - x_G|$ و $dy = |y_n - y_G|$ باشد:

$$h(n) = \max(dx, dy) + (\sqrt{2} - 1) \min(dx, dy)$$

دلیل: این تابع با در نظر گرفتن هزینه واقعی حرکت‌های قطری ($\sqrt{2}$)، نسبت به فاصله اقلیدسی یا منهتن، تخمین بسیار نزدیک‌تری به کوتاه‌ترین مسیر ممکن در شبکه ارائه می‌دهد و در عین حال به دلیل عدم دست‌بالا گرفتن هزینه، «قابل قبول» باقی می‌ماند.

(۲) بررسی قابل قبول بودن فاصله منهن با در نظر گرفتن ارتفاع: خیر، در سناریوی پرواز پهباد که تغییر ارتفاع (z) نیز هزینه دارد، فاصله منهن لزوماً قابل قبول (Admissible) نیست. دلیل: فاصله منهن تنها جابه‌جایی در صفحات افقی را محاسبه می‌کند. اگر مسیری در فضای سه‌بعدی وجود داشته باشد که از نظر افقی طولانی‌تر باشد اما به دلیل تغییرات ارتفاعی کمتر، هزینه کل (زمان یا سوخت) کمتری نسبت به مسیر مستقیم منهن داشته باشد، مقدار تخمینی منهن از هزینه واقعی (h^*) بیشتر شده و شرط $h(n) \leq h^*(n)$ نقض می‌گردد.

(۳) مفهوم آرام‌سازی (Relaxation) با حذف شرط برخورد: اگر فرض کنیم پهباد می‌تواند از درون ساختمان‌ها عبور کند (حذف مانع)، تابع هیوریستیک حاصل دارای ویژگی‌های زیر است:

- قابل قبول بودن: این تابع قطعاً قابل قبول است، زیرا کوتاه‌ترین مسیر در فضای بدون مانع همواره کوچک‌تر یا مساوی کوتاه‌ترین مسیر با وجود موانع است.
- دقت: این تابع نسبت به حالتی که موانع را در نظر می‌گیرد، قدرت کمتری دارد. حذف محدودیت‌های مسئله باعث می‌شود تخمین هیوریستیک از واقعیت دورتر شده و الگوریتم برای جبران این فاصله، گره‌های بیشتری را گسترش دهد.